

SmartRFP Pipeline – Testing Strategy

End-to-End Test Suite

Overview

This test module validates the full SmartRFP pipeline from ingestion to final outputs. It ensures that a Request for Proposal (RFP) document flows correctly through every stage of processing, including extraction, enrichment, generation, persistence, and export.

Only external dependencies (LLM and web search via Tavily) are mocked, while all internal pipeline components and database interactions are exercised end-to-end.

Key Principles

- End-to-end validation of real system behaviour
- LLM responses are mocked to ensure deterministic output
- Database is real (via temp_db fixture)
- Exports are validated for non-empty binary/text output
- Ensures all pipeline stages produce expected side effects

Integration Tests

Overview

This test module validates the SmartRFP pipeline at the integration level, ensuring that internal pipeline components interact correctly with the real database while external dependencies are fully mocked.

Unlike end-to-end tests, this suite focuses on module integration, data flow correctness, and persistence behavior rather than full system execution.

Scope of Testing

- Real Components
- SQLite database (via temp_db)
- Core pipeline orchestration (run_pipeline)
- Database persistence layer
- Mocked Components

All external and compute-heavy subsystems are replaced with deterministic fakes:

- Requirement extraction (extract_requirements)
- RAG system (RAGAgent)
- Pricing service (fetch_pricing)
- Draft generation (generate_draft)
- Pipeline evaluation (evaluate_pipeline)

Unit Tests

Database:

1. **Create and Retrieve RFP:** Verifies that an RFP can be successfully created and later retrieved with all stored metadata intact.
2. **List All RFPs:** Ensures that multiple RFPs can be stored and retrieved as a collection.
3. **Update RFP Status:** Validates that an RFP's lifecycle status can be modified after creation.
4. **Update RFP Assignment:** Ensures assignment metadata (role and owner) can be modified post-creation.
5. **Update RFP Metrics:** Validates updating analytical metadata associated with an RFP.
6. **Save and Retrieve Requirements:** Ensures extracted requirements can be stored and later retrieved in structured form.
7. **Save and Retrieve Draft Sections:** Validates persistence of generated proposal sections.
8. **Update Draft Section Content:** Ensures individual draft sections can be modified after creation.
9. **Save and Retrieve Pricing:** Validates storage and retrieval of pricing data associated with an RFP.
10. **Add and Retrieve Knowledge Base Documents:** Ensures knowledge base documents can be added and retrieved.
11. **Knowledge Base Count Accuracy:** Validates that the system correctly tracks the number of stored knowledge base documents.
12. **Audit Log Entry Creation:** Ensures that system actions are properly recorded in the audit log.
13. **RFP Deletion and Cascading Behaviour:** Ensures that deleting an RFP correctly removes all related dependent records.

Extractor:

1. **Extract "must" requirement:** Ensures that statements containing "must" are correctly identified and normalized into structured requirements.
2. **Extract "shall" requirement:** Validates that formal contractual obligations using "shall" are extracted as requirements with cleaned text.
3. **Extract "should" requirement:** Ensures softer "should" statements are still captured as valid, lower-priority requirements.
4. **Question is requirement:** Confirms that interrogative sentences (questions) are treated as implicit requirements and preserved.
5. **Section header detection:** Verifies that requirements are correctly associated with their respective document section headers
6. **Fallback when no keywords:** Ensures meaningful requirements are still generated even when explicit requirement keywords are absent.
7. **Max items limit:** Confirms that extraction respects the configured maximum limit to prevent unbounded output.

8. **LLM extract success:** Validates that structured JSON output from the LLM is correctly parsed into requirement objects.
9. **LLM extract JSON fence:** Ensures robustness when LLM responses are wrapped in Markdown code fences and still correctly parsed.

File Handler:

1. **Extract TXT file:** Ensures plain text files are correctly read and returned as clean UTF-8 strings.
2. **TXT Latin-1 encoding:** Verifies that TXT files encoded in Latin-1 are correctly decoded without data loss.
3. **Extract DOCX content:** Confirms that paragraphs in DOCX files are properly extracted and concatenated into readable text.
4. **Extract DOCX table content:** Ensures text inside DOCX tables is correctly parsed and included in the extracted output.
5. **Cleaning removes extra blank lines:** Validates that excessive newlines and blank spacing are normalized into clean, readable text.
6. **Cleaning removes multiple spaces:** Ensures repeated whitespace characters are collapsed into single spaces for consistent formatting.
7. **Unsupported file extension:** Confirms that unsupported file types (e.g., images) raise a ValueError.
8. **Legacy DOC format not supported:** Ensures old .doc files are rejected to prevent unsupported binary parsing issues.
9. **Empty text file:** Validates that empty files raise an error instead of producing invalid output.
10. **Whitespace-only file:** Ensures files containing only whitespace are treated as invalid input.
11. **Multiline text extraction:** Confirms that multi-line RFP-style text is preserved correctly while maintaining structure and content integrity.
12. **Unicode text handling:** Ensures UTF-8 encoded text with special characters is correctly decoded and cleaned without losing key content.

LLM:

1. **Demo response contains context:** Ensures the offline demo generator incorporates relevant contextual snippets from the prompt into its output.
2. **Demo response without context:** Verifies that when no context is provided, the system returns a generic but valid fallback response.
3. **Last error returns string:** Ensures the error-tracking function always returns a string for consistent observability.
4. **Used demo returns Boolean:** Confirms that the system correctly tracks whether demo mode was used.
5. **Get client without API key:** Validates that no LLM client is created when the Groq API key is missing.

6. **LLM available false:** Ensures availability check correctly reports false when no client is configured.
7. **LLM available true:** Confirms availability check returns true when a valid client exists.
8. **Chat demo mode:** Ensures chat falls back to demo mode when no LLM client is available and sets the correct internal state.
9. **Chat success:** Validates successful Groq API interaction returns generated content and disables demo mode flag.
10. **Chat exception falls back:** Ensures runtime failures during LLM calls trigger safe fallback to demo mode with a readable error response.
11. **Ping without key:** Confirms health check fails when no API key is configured.
12. **Ping client none:** Ensures ping fails gracefully when client initialization fails despite a present API key.
13. **Ping success:** Validates successful end-to-end connectivity check with a working fake LLM client.
14. **Ping exception:** Ensures ping reports failure and surfaces meaningful error messages when API calls fail.
15. **Demo response with error message:** Confirms that error messages are not leaked in raw form and are instead safely handled in demo output.

Pricing:

1. **Detect single keyword:** Ensures that the keyword extractor correctly identifies a single relevant pricing domain term from input text.
2. **Detect multiple keywords:** Validates that multiple domain-specific keywords (e.g., cloud, migration, support, security) are all correctly detected from a single input.
3. **No keywords:** Ensures that irrelevant text results in an empty keyword list, preventing false-positive pricing generation.
4. **Default pricing line:** Confirms that even without strong signals, the system generates baseline pricing entries for professional services.
5. **Cloud pricing generated:** Ensures that cloud-related input triggers relevant pricing components such as cloud infrastructure or compute services.
6. **Margin line exists:** Validates that every pricing output includes a final margin line item.
7. **Margin not stale:** Ensures that the margin line is always marked as fresh and not stale.
8. **One stale line when multiple items:** Confirms that in multi-item pricing outputs, exactly one line is marked as stale to simulate data freshness variability.
9. **Default line not stale:** Ensures baseline pricing entries are correctly marked as fresh when no external pricing signals are present.
10. **Pricing schema:** Validates that each pricing line contains all required fields including item, quantity, pricing, metadata, and freshness indicators.
11. **Total is numeric:** Ensures that computed totals are stored as numeric (float) values for safe aggregation.
12. **Source is pricing-api:** Confirms that pricing lines correctly identify their origin as the pricing API.

13. **Fetch pricing without web:** Ensures that when no external API key is present, pricing still works and web insight is disabled.
14. **Fetch pricing with mocked web:** Validates that optional web enrichment is correctly integrated when available.
15. **Margin is 18 percent:** Ensures that the final margin is correctly computed as 18% of subtotal excluding stale and margin lines.
16. **Totals are positive:** Confirms that all pricing line totals are non-negative, ensuring financial consistency and preventing invalid cost outputs.

RAG:

1. **Empty database initialization:** Ensures that when no knowledge base documents exist, the RAG agent initializes with an empty dataset and returns no results for any query.
2. **Initialization loads documents:** Verifies that the RAG agent correctly loads all available knowledge base documents from the database during initialization.
3. **Security query returns security doc:** Ensures that security-related queries retrieve the most relevant cybersecurity document from the knowledge base.
4. **Migration query returns migration doc:** Validates that migration-related queries correctly return documents related to system or data migration.
5. **Support query returns support doc:** Ensures that queries related to vendor support retrieve the appropriate telecom or support-related documents.
6. **Best match is first:** Confirms that the most semantically relevant document is ranked first in retrieval results.
7. **Top-k limit:** Ensures that retrieval respects the top_k constraint and does not return more documents than requested.
8. **Unknown query returns empty:** Validates that unrelated or nonsensical queries return no results due to low semantic similarity.
9. **Result schema:** Ensures that each retrieved document contains required fields: title, doc_type, content, and score.
10. **Score is float:** Confirms that similarity scores are consistently returned as floating-point values.
11. **Score between zero and one:** Ensures that similarity scores are normalized within the valid range [0.0, 1.0].
12. **Multiple queries are independent:** Verifies that separate retrieval calls do not interfere with each other and return distinct ranked results.
13. **Similarity threshold filters noise:** Ensures that low-relevance or noisy queries are filtered out and do not produce false-positive matches.